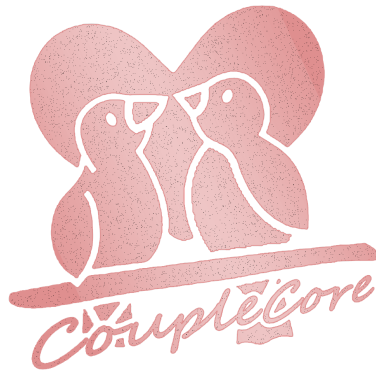




CoupleCore

Software Requirements Specification

A Relationship & Emotional Well-being Companion

Project	CoupleCore
Document	Software Requirements Specification
Version	2.0 - target specification
Platform	Flutter (iOS / Android) · Firebase
Status	Full-scope spec; current build tagged inline

Contents

Document Conventions	3
1. Introduction	3
1.1 Purpose	3
1.2 Product Scope	3
1.3 Intended Audience	3
1.4 Definitions	3
1.5 References	4
2. Overall Description	4
2.1 Product Perspective	4
2.2 User Classes	4
2.3 Operating Environment	4
2.4 Assumptions & Dependencies	4
3. System Architecture	5
3.1 Components	5
3.2 Key Data Flows	6
3.3 Privacy-Scoped Data Model	6
3.4 Security Rules & Encryption	7
4. Functional Requirements	8
4.1 Accounts & Authentication - High	8
4.2 Partner Linking - High	8
4.3 Mood Tracker - High	8
4.4 Happiness Detector - Medium	8
4.5 Emergency Alert - High	9
4.6 Shared Calendar - Medium	9
4.7 Menses Reminder - Medium (separate module)	9
4.8 Memories - Medium	9
4.9 Messaging - Medium	9
4.10 Good Deeds ("Good Collector") - Medium	9
4.11 Emotional-Support Tools - Medium	10
4.12 Notifications & Reminders - High (cross-cutting)	10
4.13 Dashboard & Navigation - High	11
5. Non-Functional Requirements	12
5.1 Performance	12
5.2 Security & Privacy - High	12
5.3 Safety - High	12
5.4 Usability & Accessibility	12
5.5 Reliability	12
5.6 Maintainability	12
5.7 Scalability & Data Efficiency	12
6. Suggested Build Roadmap	12
7. Open Issues & Decisions	13

Document Conventions

This document specifies the full target system. Each functional requirement carries a status tag reflecting the current codebase (modular Flutter app: features/, services/, models/), so the SRS is both a build specification and an honest snapshot.

Tag	Meaning
Implemented	Exists and works today.
Partial	Partly built; meaningful behaviour missing.
Placeholder	In the UI but non-functional (snackbar only).
Planned	Specified here; not yet built.

Requirement IDs: FR-n (functional), NFR-n (non-functional). Priorities are stated per feature group.

1. Introduction

1.1 Purpose

CoupleCore is a private, two-person mobile application that helps a couple strengthen communication, understand each other's emotions, avoid misunderstandings, and coordinate shared relationship activities. This SRS defines the requirements and architecture for building it as a production product.

1.2 Product Scope

The app links exactly two people into a couple. It provides authentication, partner linking, daily mood sharing, AI-assisted problem-solving tool, a multi-channel emergency alert, a shared calendar, shared memories, private messaging, a good-deeds tracker, a special days reminder. Privacy between the two partners is a core, non-negotiable product value.

1.3 Intended Audience

The development team building CoupleCore, and reviewers/testers of the product.

1.4 Definitions

Term	Meaning
Partner	The other user linked to a given account.
Couple	The linked pair; owns all shared data (couples/{coupleId}).
UID	Firebase Authentication unique user identifier.
FCM	Firebase Cloud Messaging (push notifications).
Private-by-default	Data stored only for one partner until they explicitly choose to share it.
Presence disclosure	Any signal revealing whether a partner is online or active - deliberately excluded (NFR-7).

1.5 References

CoupleCore proposal (UnP Project 01); CoupleCore Current Build Summary; requirements-interview responses (this revision); IEEE Std 830-style structure; classic “5 Love Languages” model; “How Are You Feeling?” emoji chart (mood iconography).

2. Overall Description

2.1 Product Perspective

A new, self-contained mobile product backed by managed cloud services. No third-party relationship-platform integration.

2.2 User Classes

One user class - Partner - with two symmetric instances. Both have identical rights over shared data; neither can see the other's private data unless it is explicitly shared. No admin, moderator, or external-viewer role exists by design.

2.3 Operating Environment

Client: Flutter (Material 3) app for iOS and Android.

Backend: Firebase (Authentication, Cloud Firestore, Cloud Storage, Cloud Messaging) plus Cloud Functions for server-side logic.

Connectivity: network required for auth, sync, alerts, and AI features.

Platform decision (resolved).

The proposal's web/Next.js/MongoDB stack is superseded. The confirmed target is the Flutter + Firebase mobile app described throughout this document.

2.4 Assumptions & Dependencies

Each user has exactly one active partner. Both partners install the same app. External dependencies: an LLM provider (for AI features), a telephony provider for emergency SMS/voice (e.g. Twilio), and device location services. Secrets for these live server-side in Cloud Functions, never in the client.

3. System Architecture

CoupleCore uses a client + serverless architecture. The Flutter app talks directly to Firebase for identity, data, media, and push, and delegates anything that needs a secret key or cross-user coordination (AI calls, emergency escalation, scheduled reminders, unlink cleanup) to Cloud Functions.

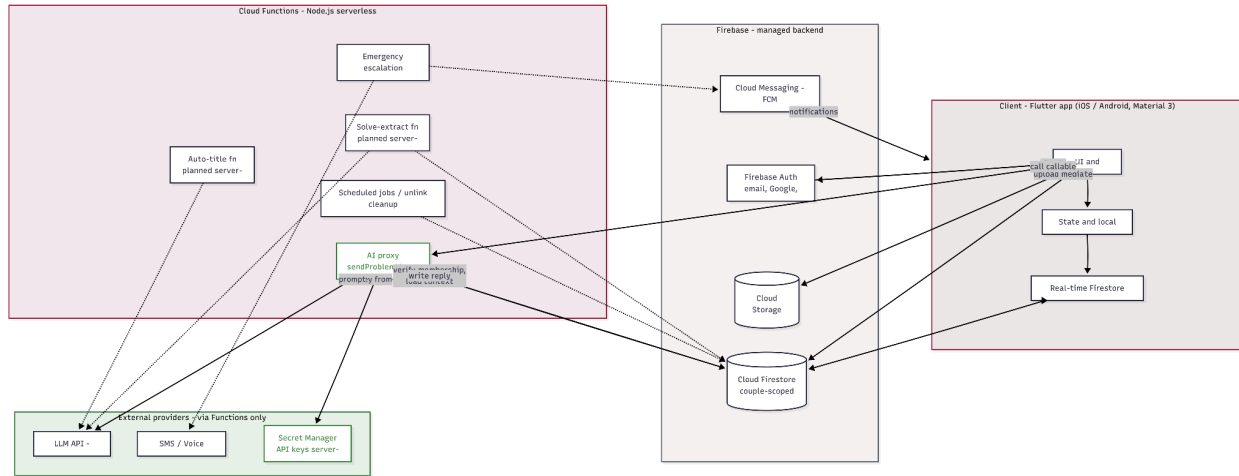


Figure 1 - CoupleCore system architecture.

3.1 Components

Layer	Component	Responsibility
Client	Flutter app (Dart, Material 3)	UI, state, local cache, real-time listeners.
Identity	Firebase Auth	Email/password, Google, Apple sign-in; password reset.
Data	Cloud Firestore	Couple-scoped documents with security rules.
Media	Cloud Storage	Photos/videos for Memories.
Push	Cloud Messaging (FCM)	Reminders, gentle nudges, emergency delivery.
Serverless	Cloud Functions (Node.js)	AI proxy, emergency escalation, scheduled jobs, unlink cleanup.
AI	LLM API (via Function)	Problem-solving tips, suggestion generation.
Telephony	SMS/Voice provider (via Function)	Emergency SMS + call fallback.

3.2 Key Data Flows

Mood: app writes to Firestore; the partner's app listens and re-renders. No write triggers a notification (NFR-7).

Problem-solving (AI): the client calls an authenticated Cloud Function (the AI proxy), which verifies couple membership, loads the conversation and the relevant solved-problem context from Firestore, calls the LLM with the key held server-side in Secret Manager, and writes the reply back. Only minimal, relevant context is sent (NFR-6). Migration is in progress: the chat-reply path is server-side; the solve-extract and auto-title calls are being moved next, after which no key remains in the client.

Emergency: app writes an alert (with location) to Firestore and invokes the escalation Function, which pushes a full-screen FCM alert and, as fallback, sends SMS and places a voice call to the partner. The partner acknowledges back into Firestore.

Unlink: a Function deletes the couple document, its subcollections, and associated media, and clears both users' link fields - severing sharing and reclaiming storage (NFR-9).

3.3 Privacy-Scoped Data Model

Data is partitioned into three scopes so that security rules are simple and privacy is enforced by structure, not by convention:

Scope	Location	Who can access
Account	users/{uid}	The owner (profile, settings, link state).
Private	users/{uid}/private/**	The owner only - encrypted; onboarding answers, private menses, sensitive notes.
Shared	couples/{coupleId}/**	Only the two members in memberUids.

Private-by-default items (e.g. onboarding Q&A) live in the owner's private scope with a per-item share flag; sharing copies/moves them into the couple scope only on explicit action.

Principal collections

users/{uid}	profile, authProviders[], loveLanguage?, partnerUid?, coupleId?, settings{ quietHours, notifPrefs }
users/{uid}/private/{docId}	onboarding answers (shared:false), private menses, sensitive fields (encrypted)
invites/{code}	fromUid, email?, status, expiresAt
couples/{coupleId}	memberUids[2], status, createdAt
couples/{coupleId}/moods/{uid}	mood, updatedAt (server-only; never surfaced)
couples/{coupleId}/moodHistory/...	daily entries, source: tracker quiz
couples/{coupleId}/calendar/{id}	title, type, start, end?, recurrence?, reminders[]
couples/{coupleId}/memories/{id}	kind: photo video note, storageRef?, text?, createdBy
couples/{coupleId}/messages/{id}	fromUid, text, kind: chat loveNote, sentAt
couples/{coupleId}/goodDeeds/{id}	byUid, description, status: pending confirmed, points

```

couples/{coupleId}/menses/{id}      predictedDate, actualDate?, visibility: shared|private,
ownerUid

couples/{coupleId}/alerts/{id}      fromUid, toUid, message, location, status, channels[],
acknowledgedAt?

couples/{coupleId}/problemSessions/{id} status: active|solved, title, lastMessage, starred?,
createdAt      ·      .../messages/{id}  role, content, sender, createdAt      ·
couples/{coupleId}/solvedProblems/{id} problem, solution, impact, sourceTitle, tags[],
createdAt (structured record; feeds the AI memory)

```

3.4 Security Rules & Encryption

Firestore rules grant read/write on couples/{coupleId}/** only when request.auth.uid is in that couple's memberUids, and on users/{uid}/private/** only to the owner. Sensitive private fields (menses, onboarding answers, personal notes) are additionally encrypted so they are unreadable even in a raw data dump. This realises the proposal's promise that no one outside the couple can see their data (NFR-5).

4. Functional Requirements

4.1 Accounts & Authentication - High

FR-1 Partial Register with email/password, Google, or Apple. (Email/password exists; social providers planned.)

FR-2 Partial Sign in with any supported provider.

FR-3 Planned Reset a forgotten password / recover the account.

FR-4 Implemented AuthGate routes signed-in users to the Dashboard, signed-out users to the Auth screen.

FR-5 Planned Manage profile and settings (notification preferences, quiet hours) from a Settings screen.

FR-6 Planned Delete the account, removing personal and shared data.

FR-7 Planned Export the user's data on request.

4.2 Partner Linking - High

FR-8 Planned Generate a single-use invite code to link a partner.

FR-9 Planned Link by entering a partner's invite code.

FR-10 Partial Link by email invitation that the partner explicitly accepts. (Current build silently resolves by email with no acceptance.)

FR-11 Planned A couple becomes active only after mutual confirmation; a couples/{coupleId} record is then created.

FR-12 Planned Onboarding Q&A during linking: answers are private by default and shared only when the owner chooses.

FR-13 Planned Unlink at any time: severs the connection, revokes all shared access, and deletes shared data/media to reclaim storage. A user has at most one active partner.

4.3 Mood Tracker - High

FR-14 Implemented Log one of six moods (Happy, Neutral, Sad, Stressed, Loved, Frustrated) via a picker.

FR-15 Implemented The dashboard shows both partners' current moods live via Firestore listeners.

FR-16 Planned Mood is logged once per day; the last logged mood persists on display until updated.

FR-17 Planned A daily reminder prompts the user to log today's mood until they do (respecting quiet hours).

FR-18 Planned A private history/trends view of the user's own moods over time.

FR-19 Planned Pattern detection over mood history, with healthy-action recommendations.

No presence disclosure (design rule, NFR-7).

Logging a mood never notifies the partner, and because the last mood persists until updated, the partner cannot infer whether the other came online today. The app exposes no online status, "last seen," or read receipts anywhere.

4.4 Happiness Detector - Medium

FR-20 Planned A daily quiz of five fixed questions captures emotional state.

FR-21 **Planned** If the result is sad, the partner receives a gentle, supportive notification that does not reveal timing or activity.

FR-22 **Planned** Quiz results feed the mood history (source: quiz).

4.5 Emergency Alert - High

FR-23 **Implemented** A floating button plus confirmation dialog sends an emergency alert addressed to the partner.

FR-24 **Planned** The alert carries a custom message and the sender's current location.

FR-25 **Critical** The partner receives and sees the alert: a full-screen notification with sound, escalating to SMS and an automated voice call if unacknowledged. (Today nothing reads the alerts data, so alerts never arrive - highest-priority gap.)

FR-26 **Planned** The partner can acknowledge/respond; status is recorded.

FR-27 **Planned** An alert history is retained.

FR-28 **Planned** A clear notice states the feature is not a substitute for emergency services, with a shortcut to call local emergency.

4.6 Shared Calendar - Medium

The calendar is the single home for all date-based events and their reminders.

FR-29 **Placeholder** Create, edit, and delete events: anniversaries, birthdays, special days, exam timetables, lecture times, discussion times.

FR-30 **Planned** Support recurring events.

FR-31 **Planned** Per-event reminders delivered by push (respecting quiet hours).

FR-32 **Planned** Both partners can view and edit shared events.

4.7 Menses Reminder - Medium (separate module)

FR-33 **Planned** Predict the cycle from previously logged dates and remind the owner.

FR-34 **Planned** When the owner records the actual date, it corrects the prediction for that entry.

FR-35 **Planned** Default visibility is shared (both partners can see and update). It can be set private, in which case only the owner sees it and it is stored in the owner's private, encrypted scope.

FR-36 **Planned** When shared, entries can be surfaced and corrected from the shared calendar.

4.8 Memories - Medium

FR-37 **Placeholder** Add photos, videos, and notes/stories.

FR-38 **Planned** Browse memories in a timeline view.

4.9 Messaging - Medium

FR-39 **Placeholder** A private couple chat (text, and optionally media).

FR-40 **Planned** Quick "love-note" sending, drafted with help from the Suggestion Generator (FR-46).

4.10 Good Deeds ("Good Collector") - Medium

FR-41 **Placeholder** Log a good deed performed for the partner.

FR-42 **Planned** The partner confirms the deed, which awards points (confirmation prevents self-inflation).

FR-43 **Planned** Show running totals and a periodic summary.

4.11 Emotional-Support Tools - High

FR-44 Planned Love-language quiz at signup using the classic five love languages; result stored on the profile.

FR-45 Planned Daily personalized suggestions tailored to each partner's love language.

FR-46 Planned Suggestion Generator produces compliments and mood-boosting notes.

Problem Solver - the AI-assisted problem-solving subsystem

The single "Problem journal + AI tips" concept in the original spec (FR-47/FR-48) has been built out into a Problem Solver subsystem. The requirements below replace the two original lines and describe both what exists today and the approved next-step design.

FR-47 Implemented Problem Solver chat: each couple holds one active AI-counsellor conversation. Messages are persisted per-message to Firestore in real time (role, content, sender, timestamp) and survive logout/app close. Replaces the earlier raw "problem journal" - problems are worked through as a live chat, not a static log.

FR-48 Implemented Mark as Solved: the AI reads the conversation and extracts a structured record - problem, solution, impact (and tags) - shown to the user for confirmation. On confirm, the record is written to solvedProblems and the raw chat is then deleted (extract → confirm → save → cascade-delete). Deletion only runs after the save is verified, so no solved insight is ever lost.

FR-48a Implemented Solved records are stored as structured documents at couples/{coupleId}/solvedProblems/{id} (problem, solution, impact, sourceTitle, tags[], createdAt) and listed in a Solved Problems view. Raw transcripts are not retained after solving.

FR-48b Implemented Normal-chat eviction: unsolved, unstarred chats are capped at the newest 5 per couple - creating a 6th hard-deletes the oldest (with its messages cascade-deleted). Solved and starred chats are excluded from the count and never auto-deleted. Keeps storage lean without losing anything the couple chose to keep.

FR-48c Implemented Star to keep: any chat can be starred to protect it permanently from the newest-5 eviction. Starred chats are excluded from both the keep-5 count and deletion.

FR-48d Implemented Chats list: a ChatGPT-style list of surviving chats (newest 5 normal + all starred), each openable to full history with a working input so the conversation can be continued from the last message.

FR-48e Partial Auto-titles: a chat is given a short descriptive title generated from its first user message (falling back to a date title). Implemented client-side, but temporarily disabled while the AI call is being moved server-side (see architecture note); titles currently fall back to the date form until ported.

FR-48f Planned Solved-memory retrieval (approved design): when answering, the AI should draw on the couple's solved history so advice is personalised. Two rules apply - (1) memory is consulted only when the user is actually asking for a solution, detected by a lightweight keyword/pattern gate on the incoming message, so ordinary chat does no memory lookup; (2) when consulted, the whole solved history is searchable but only the records relevant to the current question are retrieved (by tag/keyword overlap against solvedProblems) and sent to the LLM - not the entire history. This keeps replies fast and cheap as the solved history grows.

Interim behaviour: the deployed reply path currently fetches the most recent solved records and includes them on every message. This is a stopgap; FR-48f (keyword-gated, relevance-scoped retrieval over all solved records) is the target and supersedes it

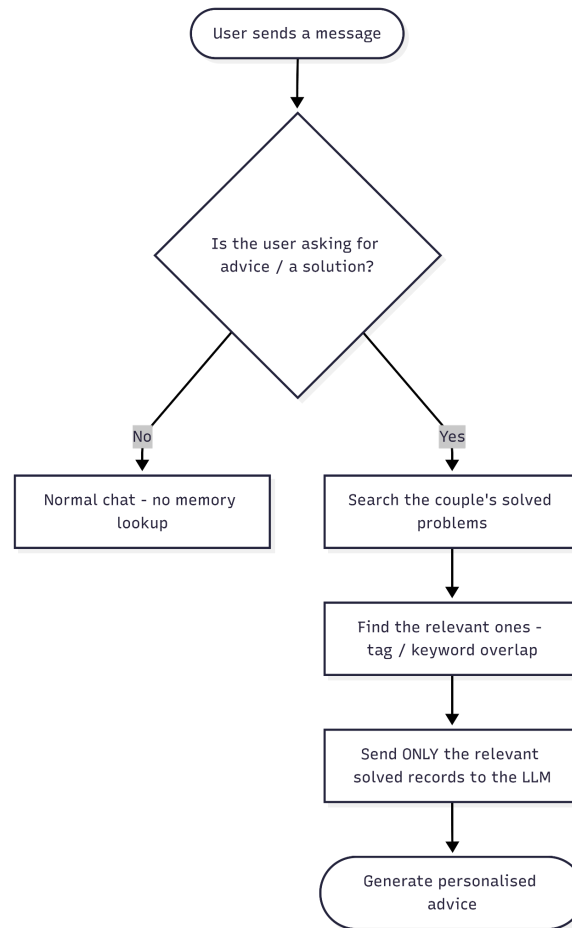


Figure 2 - FR-48f: solved-memory retrieval logic (keyword-gate + relevance search)

4.12 Notifications & Reminders - High (cross-cutting)

FR-49 **Planned** Push delivery via FCM. (firebase_messaging is a declared dependency but not yet wired up.)

FR-50 **Planned** User-configurable notification preferences and quiet hours; reminders are suppressed during quiet hours.

FR-51 **Planned** Triggers: upcoming calendar events, the daily mood-log nudge, a sad happiness-detector result, emergency alerts, and menses reminders.

4.13 Dashboard & Navigation - High

FR-52 **Implemented** Dashboard with header (day/date), mood section, and feature grid.

FR-53 **Placeholder** The “Wisdom” card presents rotating content driven by data/date (currently static).

FR-54 **Partial** Bottom navigation switches between real Home, Timeline, Alerts, and Settings screens (currently only the icon highlight changes).

5. Non-Functional Requirements

5.1 Performance

NFR-1 Mood, message, and alert updates propagate to the partner in near-real-time via snapshot listeners. NFR-2 The app stays responsive on a mid-range device; media loads progressively.

5.2 Security & Privacy - High

NFR-3 All traffic is encrypted in transit (TLS). NFR-4 Credentials are handled by Firebase Auth (secure storage, token issuance). NFR-5 Firestore security rules restrict shared data to the two couple members and private data to its owner. Sensitive private fields are additionally encrypted.

NFR-6 AI processing sends only the minimal necessary context to the LLM provider; keys stay server-side, and the provider's data-handling terms are reviewed before launch. NFR-8 Private-by-default: personalized/sensitive data is stored per-partner and shared only on explicit action.

NFR-7 No presence disclosure.

The app must never reveal whether a partner is online, recently active, or has opened the app today - no presence dots, "last seen," typing or read indicators. Mood persistence and the gentle-notification wording are designed to uphold this.

5.3 Safety - High

NFR-10 The emergency feature is presented as a private signal to the partner, not a replacement for emergency services (FR-28). NFR-11 Emergency escalation (push → SMS → call) must be reliable and its delivery status observable; the feature must not be exposed as "working" until end-to-end delivery exists.

5.4 Usability & Accessibility

NFR-12 Core actions (log mood, send love-note, trigger emergency) take one or two taps. NFR-13 UI follows Material 3 conventions for consistency and accessibility.

5.5 Reliability

NFR-14 The app degrades gracefully offline with clear messaging and syncs when reconnected.

5.6 Maintainability

NFR-15 Done: the original single-file build has been refactored into modular layers (features/, services/, models/). Maintain this structure as features grow.

5.7 Scalability & Data Efficiency

NFR-16 Data is partitioned per couple, so it scales horizontally. NFR-9 Storage efficiency is a first-class concern: avoid duplication, cap/compress media, and delete shared data and media on unlink so no orphaned couple data remains.

6. Suggested Build Roadmap

A phased path from the current build to the full spec, front-loading the architecture the rest depends on.

Phase	Focus	Key requirements
1 - Foundation	Auth providers + reset, robust linking (invite code / email accept / mutual confirm), couple data model + security rules, settings & quiet hours.	FR-1–3, 5, 8–13; NFR-5
2 - Core emotional	Mood history & patterns, happiness detector, full emergency pipeline (receive + location + escalation), FCM notifications.	FR-16–28, 49–51; NFR-7, 10, 11
3 - Shared life	Calendar + reminders, memories, messaging, good deeds, menses module.	FR-29–43
4 - AI & polish	Problem journal + AI tips, love-language suggestions, suggestion generator, dashboard/nav completion.	FR-44–48, 53–54; NFR-6

7. Open Issues & Decisions

Item	Detail
Emergency delivery (critical)	Build FR-25 end-to-end before exposing the feature. Choose the telephony provider (SMS + voice) and budget for per-message/call costs. Note platform constraints: Android full-screen intents and iOS critical-alert / CallKit entitlements.
AI provider & privacy	Select the LLM provider; review its data-retention terms; minimize/anonymize the problem data sent (NFR-6).
Encryption scheme	Decide client-side field encryption vs. rules-only for sensitive private data, and key management per couple.
Gentle-notification wording	Must support the partner without revealing activity timing, to uphold NFR-7.
Good-deeds & messaging scope	Confirmed as partner-confirmed points and chat-plus-love-notes respectively; refine rewards/summary cadence during Phase 3.
Migration	Move current flat data (users/{uid}, moods/{uid}, alerts) into the couple-scoped model in Phase 1.

End of document - CoupleCore SRS v2.0, target specification synthesized from the proposal, current build, and requirements interview.